ChatGPT

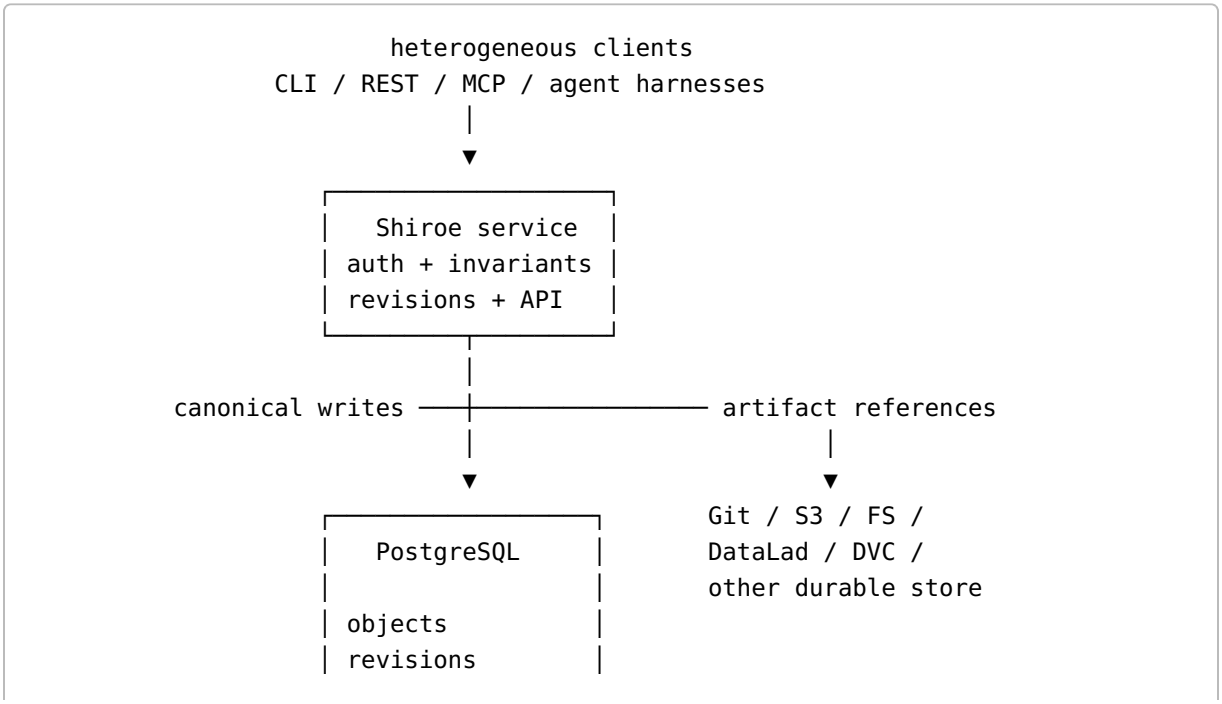
Shiroe Core Architecture Survey: Phase 0

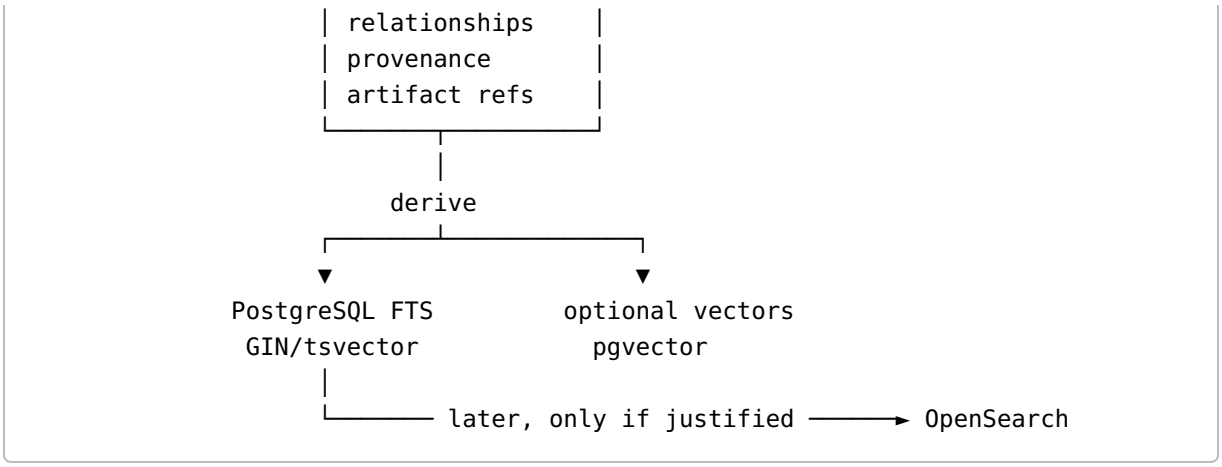
Executive assessment

Shiroe's own requirements sharply constrain what Phase 0 should be. The authoritative plan says persistent knowledge belongs to Shiroe rather than an LLM session; important objects need stable identity and provenance; observed, human-authored, derived, inferred, and speculative knowledge must remain distinguishable; large artifacts may remain external; heterogeneous clients must go through documented interfaces; Git is important but must not become the Shiroe database; and schemas must be allowed to evolve. filecite turn0file0 The vision further frames Shiroe as a connective research-state layer spanning literature, projects, and experiments rather than as another document library, experiment tracker, or agent-local memory mechanism. filecite turn0file1

The tentative PostgreSQL + JSONB + PostgreSQL full-text-search architecture survives this survey, with an important qualification: PostgreSQL should be the canonical transactional store, not a giant JSON document bucket. The strongest pattern across mature research repositories and ML tracking systems is to keep authoritative metadata and identity in a transactional database while separating large artifacts and, where justified, disposable search projections. Dataverse uses PostgreSQL for relational state, Solr for search, and separate file storage; MLflow similarly separates its backend metadata store from artifact storage; Invenio separates JSON record storage with revision history from pluggable search/database components. [1]

For Shiroe, the minimal core should therefore be:





That design is not merely the “fewest services” answer. PostgreSQL directly supplies primary/foreign-key integrity, many-to-many relationships, transactions and concurrent access, indexed JSONB, and GIN-indexed full-text search. PostgreSQL 18 is the current stable documentation line as of October 2026, while PostgreSQL 19 remains beta. [2] Its permissive PostgreSQL License imposes no commercial-edition requirement for these capabilities. [3]

The main correction to the tentative plan is conceptual: **JSONB should absorb schema variability, not identity, referential integrity, provenance, or revision semantics.** PostgreSQL's JSONB is efficient, indexable, and particularly useful where fields are evolving, but its own documentation notes that JSON documents work best as atomic data whose internal pieces do not need independent modification. Shiroe's IDs, revision pointers, evidence edges, derivations, actors, and artifact identities *do* need independent integrity and queryability, so those belong in conventional columns and tables. [4]

A second correction is that **provenance should be first-class rather than buried in arbitrary object JSON**. W3C PROV's durable architectural lesson is the separation of entities, activities, agents, derivations, generation/use, attribution, and bundles; it also explicitly provides concepts for specialization and alternate versions. [5] Shiroe need not adopt RDF or the complete PROV ontology, but a small PROV-inspired relational model is a much better foundation than free-form `"provenance": {...}` blobs.

A third finding is that **agent memory products are useful precedents but poor owners of Shiroe's truth**. Letta demonstrates persistent state shared across sessions and clients and lets independently addressable memory blocks be shared among agents; its current architecture explicitly separates agent execution/state from durable application/product state. [6] Mem0 demonstrates multi-signal semantic/lexical/entity retrieval, but its April 2026 architecture treats agent-generated facts as first-class memories with equal weight and its managed platform contains proprietary optimizations absent from the open-source SDK. That is nearly the opposite of Shiroe's requirement to preserve epistemic distinctions. [7]

The Phase 0 recommendation is consequently:

Adopt PostgreSQL as the sole required persistence service. Build Shiroe's own small revision/provenance envelope. Integrate Git and external artifact stores by immutable references rather than ingestion. Expose one application service through REST, with CLI and MCP adapters over the same semantics. Start lexical retrieval in PostgreSQL. Make pgvector optional. Defer OpenSearch, a graph

database, Redis, and dedicated agent-memory infrastructure until a prototype demonstrates a capability or scaling failure that actually requires them.

That is consistent with the project's stated completion gate—multiple heterogeneous clients reaching the same persistent knowledge through stable interfaces—without prematurely solving the much larger Phase 1–3 questions. filecite turn0file0

Representative architecture comparison

The matrix below separates **documented behavior** from **Shiroe architectural judgment**. "Unknown" means the reviewed primary sources did not establish the capability; it should not be read as "unsupported."

Representative	Documented persistence / identity / recovery	Maintenance, license, hosted boundary	Material limitations and Shiroe judgment
Dataverse — research repository	Dataverse's architecture places relational state in PostgreSQL and search in Solr, with persistent identifiers handled through PID services; production deployments may place physical files in S3 separately from database and search. [8] Dataverse documents explicit backup requirements for durable file content. [9]	The project remained active in 2026, with a 6.12 milestone/release activity in September 2026. [10] **Core license:** unknown from the primary pages reviewed here.	**Precedent, not adoption.** Strong evidence for canonical DB + derived search + external artifacts. Its repository/publication semantics are too specialized to become Shiroe's generic research-state model. Detailed epistemic distinctions are **unknown** from the architecture sources reviewed.
Invenio / InvenioRDM — scientific repository framework	`invenio-records` stores JSON documents identified by unique UUIDs and explicitly provides revision history and JSON Schema validation. [11] InvenioRDM's record layer is DataCite-based and supports PostgreSQL/MySQL plus OpenSearch-backed configurations. [12]	MIT licensed; core RDM modules were actively updated in October 2026. [13] No required paid edition was identified for the capabilities evaluated.	**Excellent precedent for "flexible record + stable envelope + revisions."** It is nevertheless a repository framework with substantial domain machinery Shiroe does not yet need. Index reconstruction behavior was **not verified** in the reviewed sources, so it should not be assumed.

Representative	Documented persistence / identity / recovery	Maintenance, license, hosted boundary	Material limitations and Shiroe judgment
MLflow Tracking — experiment client/server	Tracking metadata such as run/model/trace IDs, parameters, metrics, and tags can live in a relational backend; large artifacts live in a separate artifact store. Production documentation recommends a database backend such as PostgreSQL/MySQL for higher concurrency, while the old file backend is legacy. [14] Artifact access can be proxied through the server or sent directly to object storage. [15] Database schema upgrades use explicit migrations, and MLflow warns migrations can require downtime or be non-invertible and recommends a backup first. [16]	Apache-2.0 and very actively maintained; the main repository was updated on October 12, 2026. [17] The surveyed tracking architecture itself is open source; commercial managed MLflow offerings are not necessary for these fundamentals.	**Strong precedent and future integration target, not Shiroe's DB.** The separation of metadata and artifacts is almost exactly right. Its canonical object model remains experiment/run-centric, and no generic distinction among human assertions, observations, model interpretations, and hypotheses was established in the reviewed tracking docs.

Representative	Documented persistence / identity / recovery	Maintenance, license, hosted boundary	Material limitations and Shiroe judgment
Aim — experiment tracker	Aim uses a custom RocksDB-based repository and offers a FastAPI/ uvicorn remote tracking server so multiple hosts can centralize experiment data over HTTP/ WebSocket. [18] Its CLI includes a `reindex` operation and storage-maintenance commands. [19]	The repository identifies Aim as Apache-2.0 and offers enterprise support beyond core Aim; which concrete features are enterprise-only is **unknown** from the reviewed primary material. [20] An open September 2026 repository issue reports an unauthenticated remote-tracking RPC deletion path; that report should be treated as unresolved rather than as a confirmed advisory until maintainers resolve it. [21]	**Precedent only.** Attractive single-purpose remote tracking, but a custom storage engine creates an unnecessary second source of truth for Phase 0, and current security/operational questions make adopting its server for Shiroe particularly unattractive.
Letta — stateful agents / memory	Letta agents persist identity and memory across conversations and, in cloud-backed operation, across machines/interfaces. [22] Its App Server supports multiple agents and concurrent clients; its architecture documentation says durable product/ application state should remain owned by the application/controller rather than being inferred from the agent stream. [23] Memory blocks have stable IDs and can be shared among agents independently of any single agent. [24]	The current Letta ecosystem is active, with core SDKs/Letta Code updated in October 2026; major current components are Apache-2.0. [25] Cloud-hosted persistence is available; local/self-hosted operation also exists. [22]	**Integrate later if useful, never canonicalize Shiroe state in it.** The strongest transferable idea is *stable shared memory objects behind a service*, plus explicit sequence/ idempotency concepts. [26] Evidence provenance and Shiroe's epistemic classes are **unknown/not established** by the reviewed memory-block model.

Representative	Documented persistence / identity / recovery	Maintenance, license, hosted boundary	Material limitations and Shiroe judgment
Mem0 — agent memory layer	The April 2026 memory algorithm is additive rather than update/delete oriented, performs entity linking, and combines semantic, BM25 keyword, and entity signals. It now treats confirmed agent-generated facts as first-class memories. [7]	Apache-2.0 and active as of October 2026. [27] The project's own README says its published benchmark numbers reflect the **managed platform**, which contains proprietary optimizations unavailable in the open-source SDK. [7] Its former MCP repository was archived in March 2026. [28]	**Retrieval precedent, reject as canonical memory.** Treating agent-generated “facts” with equal memory weight is incompatible with Shiroe's requirement that interpretation/inference remain distinguishable from observations and human knowledge. The archived MCP bridge is also not an integration foundation to adopt.
W3C PROV — provenance standard	PROV-DM defines Entities, Activities and Agents plus relations including usage, generation, derivation and attribution; bundles permit provenance about provenance, while specialization/alternate relations can express aspects and variants. [5]	Stable W3C Recommendation rather than a deployed product; paid/hosted edition is not applicable. [29]	**Adopt the pattern, not the whole representation.** It solves “how was this thing produced and by whom/ from what?” but does **not by itself provide Shiroe's observation/ assertion/ interpretation/ hypothesis taxonomy**. Add that as a small Shiroe-level field rather than pretending provenance and epistemic status are the same thing.

Representative	Documented persistence / identity / recovery	Maintenance, license, hosted boundary	Material limitations and Shiroe judgment
Wikibase — statement-oriented knowledge base	Wikibase entities have canonical identifiers; its JSON representation includes entity revision information, and statements carry their own identifiers, ranks, qualifiers, and references. References are explicitly intended to record provenance/authority information. [30] Save-time constraints are available. [31]	Wikimedia's Wikibase ecosystem remains active in 2026. [32] **Exact core-extension license:** unknown from the reviewed core mirror page; related current Wikibase extensions use GPL-family licensing, but that is not sufficient evidence to assert the core license. [33]	**Very useful precedent, reject as Phase 0 substrate.** "Statement + qualifier + reference + revision" is worth copying conceptually. MediaWiki/ Wikibase brings a large application stack and a knowledge-base data model before Shiroe has established its ontology.
PostgreSQL — canonical relational + lexical store	Primary/foreign keys give stable row identity and referential integrity, including self-references and many-to-many relationships. [34] JSONB offers structured, indexed flexible documents and GIN indexes for containment/path queries. [35] Full-text search provides `tsvector` / `tsquery`, ranking/highlighting infrastructure, with GIN the preferred FTS index type. [36] PostgreSQL MVCC/locking provides shared transactional access, and `pg_dump` / `pg_restore` provide logical backup/restore. [37]	PostgreSQL 18 is current stable; permissive PostgreSQL License. [38] No paid edition is required.	**Adopt.** Best Phase 0 fit because relationships do not imply graph-database semantics and flexible metadata does not imply schemaless storage. PostgreSQL does *not* supply Shiroe's revision/provenance/ epistemic semantics automatically; those must be modeled. Its text search also should not be assumed to match all Lucene/OpenSearch relevance features.

Representative	Documented persistence / identity / recovery	Maintenance, license, hosted boundary	Material limitations and Shiroe judgment
pgvector — vector extension	pgvector adds exact and approximate vector similarity search to PostgreSQL, including HNSW and IVFFlat, and upgrades through PostgreSQL's extension mechanism. [39]	Current project code uses the PostgreSQL-style license and remained active in 2026. [40] No hosted edition is required.	**Integrate only after evidence.** It preserves the one-database operational model and transactional association between embeddings and canonical IDs. Embeddings and ANN indexes should remain explicitly **derived state** carrying model/version metadata; they must never become canonical knowledge.
OpenSearch — dedicated lexical/vector search	OpenSearch provides Lucene-backed full-text queries including fuzzy and multi-field behavior, plus vector search and hybrid pipelines combining lexical and semantic results; current hybrid search supports both score normalization and reciprocal-rank fusion. [41] Snapshots back up indexes/cluster state and the Reindex API can rebuild into new indexes, including cross-cluster workflows. [42]	Apache-2.0; OpenSearch 3.9.0 was released September 29, 2026 and the project is actively maintained. [43] The capabilities discussed here are in the open-source project; managed OpenSearch hosting is optional.	**Defer.** It becomes justified when Shiroe has demonstrated a need for advanced analyzers/ fuzziness/BM25-style multi-field search, search-specific scaling, or hybrid/vector facilities that materially outperform the PostgreSQL path. Even then it should be a reconstructable projection, never the authority.

Representative	Documented persistence / identity / recovery	Maintenance, license, hosted boundary	Material limitations and Shiroe judgment
DataLad / git-annex — distributed artifact metadata	A DataLad dataset is a Git repository, usually with a git-annex. It distinguishes a persistent dataset UUID, per-clone annex UUID, Git commit IDs for versions, and content IDs that are normally checksums independent of filename. [44] DataLad can preserve URLs and availability metadata even when file contents are absent locally. [45]	MIT/Expat licensed and actively maintained in 2026. [46] No central hosted service is required.	**Integrate opportunistically.** This is the strongest precedent for separating logical dataset identity, version identity, clone/location identity, and content identity. Shiroe should be able to record these IDs when a project already uses DataLad, but adopting DataLad globally would impose a Git-annex workflow on projects that do not need one.
DVC — Git-backed artifact metadata	DVC stores small `.dvc` / `dvc.yaml` metadata under Git while actual content lives in caches/ remotes such as S3, GCS, Azure, SSH, HDFS or filesystems. [47] Git history therefore versions artifact pointers, and DVC records external origin/ revision information for dependencies. [48] Remote data can be pushed/pulled and garbage-collected according to referenced Git/DVC history. [49]	Current official documentation is maintained. **Core license was not independently verified from a primary license page in this review, so it is recorded as unknown here.** DVC Studio provides additional registry-oriented UX, but its commercial boundary was not investigated because that is outside Phase 0. [48]	**Pattern/integration, not database.** Excellent illustration of "small versioned metadata + external content," exactly the boundary envisioned by Shiroe, but its Git-centric concurrency/ query model is unsuitable for shared live research state.

Representative	Documented persistence / identity / recovery	Maintenance, license, hosted boundary	Material limitations and Shiroe judgment
MCP — agent-facing protocol	The July 28, 2026 MCP specification defines a JSON-RPC-based protocol with version negotiation and capabilities including tools, resources and prompts. Cross-request state is not implicit: persistent state needs explicit identifiers. [50]	July 28, 2026 is the current stable specification release identified in the reviewed sources. [51] Licensing is in transition under the Linux Foundation: new specification/code contributions are moving to Apache-2.0 while documentation uses CC-BY 4.0; older material may still carry MIT terms. [52]	**Integrate as an adapter.** MCP should expose Shiroe's application semantics to agent harnesses, not own those semantics or persistence. REST/client-library/MCP paths should all call the same service layer.

Several cross-system conclusions are unusually consistent.

First, **canonical metadata and search are separate concerns even when physically implemented in one database**. Dataverse makes the separation explicit through PostgreSQL versus Solr, while PostgreSQL itself allows the same logical separation to remain inside one server through canonical tables versus GIN/`tsvector` projections. [53] The latter is the right Phase 0 optimization: preserve the boundary without paying for an extra service.

Second, **large payloads should have identity without requiring database residence**. MLflow's backend-store/artifact-store split, DVC's Git-pointer/remote-content split, and DataLad's distinction between dataset/version/content/location IDs all reach this conclusion independently. [54] This closely matches Shiroe's own explicit requirement that large datasets and artifacts may remain elsewhere. filecite turn0file0

Third, no reviewed system gives Shiroe's required epistemic distinction “for free.” W3C PROV describes how things were produced, Wikibase gives references and qualifiers, MLflow describes runs, and agent memories store learned/fact-like state, but **provenance is not the same dimension as epistemic status**. [55] Shiroe has to own that small distinction itself.

Reusable architectural patterns

Canonical core, disposable projections

The most important reusable pattern is:

> **One authoritative state model; zero or more reconstructable retrieval projections.**

For Shiroe, “canonical” should mean that losing the data would destroy information that cannot simply be recalculated. IDs, object revisions, human edits, evidence links, provenance activities, relationships, source

revision identifiers, and external-artifact locators belong here. Full-text vectors, embedding vectors, ANN structures, OpenSearch documents, result caches, snippets, and ranking features do not.

This is more precise than “PostgreSQL is canonical, OpenSearch is derived.” Even with PostgreSQL alone, a `tsvector` column or GIN index should be considered derived from canonical revision content. PostgreSQL supports rebuilding indexes while retaining the source data, and OpenSearch's own architecture likewise provides explicit reindex workflows for rebuilding search representations. [56]

Every derived projection should therefore carry enough information to answer:

```
derived_from_object_revision
derivation_kind
algorithm_or_model
algorithm_version
configuration_digest
created_at
index_generation
```

A vector without the embedding model/version is incomplete provenance. A search document without its source revision is unsafe because an agent cannot know whether a hit corresponds to the object's current state.

Stable object identity separated from revisions and aliases

Invenio's UUID + revision-history arrangement and DataLad's explicit separation among dataset identity, commit identity, clone identity, and content identity demonstrate why one identifier cannot safely mean all of these things. [57]

Shiroe should give an important logical object one stable opaque identifier for its lifetime and give each revision a distinct revision identifier. Human-friendly names such as `project:flux-fiction` should be aliases, not the sole primary key.

Conceptually:

```
logical object
    shiroe_id = stable forever

revision
    revision_id = immutable
    object_id   = stable object
    revision_no = monotonic within object
    schema_version
    payload
```

Relationships that are scientifically meaningful should be able to target either the logical object or, where evidence requires precision, an exact revision. “Project A concerns Paper P” can usually target an object; “this interpretation was derived from version 3 of source S” must target the revision.

This also prevents a subtle provenance failure: an evidence edge to a mutable “current” record is not adequate historical evidence.

Provenance as activity, evidence as linkage, epistemic class as a separate axis

A minimal PROV-inspired pattern should distinguish three concepts rather than collapsing them:

```
Entity/revision  ──used by──► Activity ──generated──► Entity/revision
                                  │
                                Agent
```

W3C PROV directly supports the entity/activity/agent and use/generation/derivation pattern. [29] Shiroe should then add an orthogonal field on knowledge-bearing revisions, for example:

```
epistemic_kind =
    observation
    human_assertion
    derived_result
    model_interpretation
    hypothesis
```

The exact names can change during later specification work; what should freeze in Phase 0 is the **ability to represent the distinction**, not an elaborate scientific ontology. This follows directly from the project's requirement that LLM interpretation remain distinguishable from observed, derived, human-authored, and speculative content. filecite turn0file0

A model interpretation could therefore say:

```
revision R91
epistemic_kind = model_interpretation

generated_by:
    activity A44

A44:
    actor = model:gpt-...
    operation = interpret
    model_version = ...
```

```
    prompt/template digest = ...
    inputs = [revision R27, revision R51]
```

An observation from an experiment can use the same provenance machinery but a different `epistemic_kind`. The storage model does not need to know what an HPC observation *means* yet.

Versioned references to external artifacts

DataLad shows particularly clearly that identity of a dataset, a version, a particular repository clone, and a file's content checksum are distinct. [44] DVC likewise leaves large content outside Git while keeping metadata sufficient to locate and recover versions. [58]

Shiroe should generalize this pattern into an artifact reference:

```
ArtifactRef
    id
    uri
    scheme/provider
    content_digest       nullable only when impossible
    size                 optional
    media_type           optional

    source_version:
        git_commit       optional
        git_blob         optional
        external_version optional

    retrieved_at         optional
    availability_state   optional
```

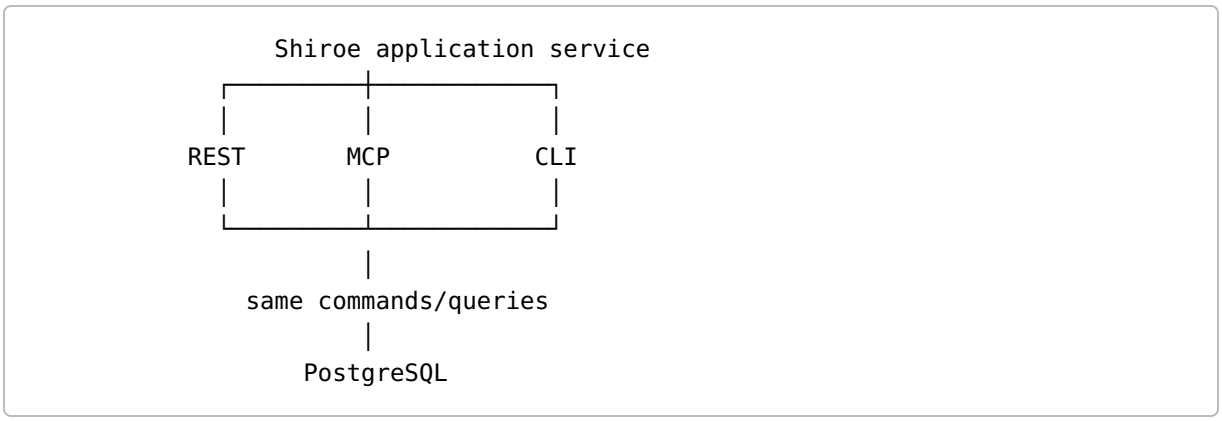
The locator and the digest serve different purposes. A URI says where something may be found; a digest says which bytes Shiroe meant. Git-backed artifacts can additionally retain repository URI, commit, path, and blob identity.

This makes S3, a parallel filesystem, DataLad, DVC, Git, DOI-backed resources, and later experiment artifact stores all possible without Phase 0 selecting one mandatory storage product.

Server ownership and protocol adapters

The project explicitly requires a central network-accessible service rather than clients directly manipulating storage. filecite turn0file0 MLflow demonstrates the value of a remote tracking endpoint separating clients from the backend/artifact topology, and Letta similarly places persistent agent behavior behind an always-on service. [59]

Shiroe should consequently have **one business-logic boundary**:



The CLI should normally be a client, not a privileged database editor. MCP should translate MCP tools/resources into the same operations used by REST rather than growing a parallel persistence implementation. MCP's explicit state identifiers fit this model well. [60]

For writes, the service—not PostgreSQL credentials distributed to laptops or HPC login nodes—owns validation, revision creation, authorization, idempotency, and optimistic concurrency. PostgreSQL supplies the transactional machinery underneath. [61]

Restore canonical state first, reconstruct everything else

Operational backup and portable export should be different mechanisms.

For operational recovery, PostgreSQL provides logical dump/restore tooling; `pg_dump` supports consistent snapshots and parallel dump formats, while `pg_restore` supports controlled and transactional restoration. [62] At larger production scale Shiroe may later require physical backups/PITR, but Phase 0 need not solve that before its actual durability requirements are measured.

For product portability, Shiroe should separately support a versioned logical export, for example:

```
manifest.json
objects.ndjson
revisions.ndjson
relations.ndjson
activities.ndjson
evidence.ndjson
artifact_refs.ndjson
```

The export should contain stable IDs, revision IDs, schema versions, provenance, and artifact references but **not require FTS indexes, ANN indexes, or OpenSearch snapshots**. Those are recoverable projections. OpenSearch itself supports snapshots, but making an external search snapshot necessary to restore Shiroe would violate the canonical/derived boundary. [63]

That gives two independent guarantees:

```
database backup       → operational disaster recovery
logical Shiroe export → implementation-independent portability

either canonical path
        ↓
rebuild FTS / embeddings / external search
```

Adopt, integrate, build, and reject

Adopt now

PostgreSQL as the only mandatory persistent service is the strongest Phase 0 decision. The combination of referential constraints, JSONB, GIN indexing, full-text search, transactions, concurrency, and mature backup/restore covers the actual core requirements without introducing multiple consistency domains. 64

Use conventional relational columns for invariants and JSONB for evolving type-specific payloads. A useful rule is:

```
If Shiroe must enforce it, join on it frequently,
refer to it from another object, or preserve it for provenance:
    make it relational.

If it is type-specific, experimental, sparse, or still evolving:
    JSONB is appropriate.
```

This exploits JSONB's indexed flexibility without surrendering relational integrity. PostgreSQL supports GIN indexing over both JSONB operators and `tsvector`, so the same database can initially serve structured filtering and lexical search. 65

PostgreSQL FTS should be the Phase 0 lexical baseline, not because it is universally the best search engine, but because Phase 0 has not demonstrated any requirement that outweighs a second distributed service. PostgreSQL already supports parsing, dictionaries, query construction, ranking, highlighting and GIN indexing. 66

Integrate behind narrow adapters

MCP should be integrated as a protocol adapter because it directly serves the heterogeneous-agent requirement. It should sit over the Shiroe application API rather than becoming the API's semantic definition or state store. 50

Git should be integrated through references, preserving repository, commit and where appropriate blob/path identity. This honors the project's deliberate division: Git owns code and human-authored version history while Shiroe owns semantic/research state and cross-project retrieval. filecite turn0file0

DataLad and DVC should be optional artifact adapters, not dependencies. DataLad is particularly compelling for HPC/research environments because it supplies stable dataset IDs plus content-availability metadata independently from payload presence; DVC provides an analogous pointer-to-remote pattern. [67] Projects already using either should be able to register their native identities rather than copying content into Shiroe.

pgvector should be a feature-gated extension, not installed because "AI means vectors." It becomes attractive once Phase 1 demonstrates that semantic retrieval materially improves Shiroe's actual query workload, because it then adds vector search without creating another consistency boundary. pgvector provides both exact and HNSW/IVFFlat approximate search inside PostgreSQL. [68]

Build the thin Shiroe-specific layer

No surveyed component provides the combination Shiroe actually needs, so the irreducibly custom surface should remain small:

1. a stable object/revision envelope;
2. first-class relationships;
3. provenance activities and source-revision references;
4. an epistemic classification independent of provenance;
5. external artifact references;
6. API-level concurrency/idempotency rules;
7. derived-index generation tracking;
8. portable logical export.

This is not an attempt to build a new database or knowledge-graph engine. It is the small semantic layer that prevents PostgreSQL rows, Git commits, agent memories, and future experiment records from becoming indistinguishable bags of metadata.

Reject or defer for Phase 0

Do not adopt InvenioRDM, Dataverse, Wikibase, MLflow, Aim, Letta, or Mem0 as Shiroe itself. Each proves useful architectural ideas, but each makes a different domain concept primary: repositories/publications, statements/wiki entities, experiment runs, or agent memories. [69] Adopting one wholesale would make later Shiroe layers fit somebody else's ontology.

Do not add OpenSearch in Phase 0. OpenSearch genuinely supplies capabilities beyond basic PostgreSQL FTS—Lucene-backed full-text behavior, fuzziness, rich multi-field queries, dedicated vector search and configurable hybrid fusion. [70] Those are reasons to add it *when needed*, not reasons to operate a search cluster before a Shiroe corpus and retrieval benchmark exist.

Do not add a graph database merely to store relationships. PostgreSQL already represents self-references and many-to-many edges with enforced referential integrity. [34] A graph service becomes justifiable only when Shiroe develops important graph-traversal workloads that relational recursive queries cannot meet at the required latency/scale. Phase 0 currently contains no such evidence.

Do not use Git, DVC, or DataLad as the live Shiroe database. Their versioned/distributed artifact semantics are valuable, but the project requires centrally shared state and concurrent heterogeneous clients. The DVC pattern itself puts metadata history in Git while content lives elsewhere; DataLad is explicitly decentralized around Git/git-annex. [71] Those properties complement a Shiroe service rather than replacing one.

Do not add Redis or a job system yet. MLflow is instructive here: shared job infrastructure such as Redis appears when multiple server replicas or asynchronous shared work require coordination. [72] Shiroe should cross that bridge when it actually introduces multi-replica background indexing or durable asynchronous jobs.

Proposed minimal persistence foundation

The following is an architectural proposal rather than a claim about existing software.

Explicit assumptions

Assumption A — Phase 0 scale is modest. One PostgreSQL instance can hold Shiroe's metadata and serve initial retrieval without sharding.

Assumption B — canonical Shiroe records are much smaller than raw research artifacts. Large PDFs, traces, datasets, binaries, and result bundles remain external, consistent with the project plan. 🗄filecite🗄turn0file0🗄

Assumption C — relationships matter immediately, but arbitrary high-depth graph analytics do not. Most Phase 0 queries are expected to be object lookup, filtered search, one/few-hop relationships, provenance traversal, and revision lookup.

Assumption D — schema evolution will be frequent. The envelope must therefore remain small and stable while type-specific payloads evolve in JSONB, again matching the project's explicit preference for migrations and stable IDs over a prematurely frozen ontology. 🗄filecite🗄turn0file0🗄

Assumption E — PostgreSQL is authoritative. Git, object stores, and external scientific systems remain authoritative for the artifact bytes they own, but Shiroe's interpretation of those artifacts—relationships, revisions, assertions, provenance—is persisted in PostgreSQL.

Minimal relational envelope

A Phase 0 schema need not know about `Paper`, `Experiment`, `Hypothesis`, or any later domain ontology yet.

```
object
------
id                    stable Shiroe ID
kind                  evolving logical type label
namespace             optional ownership/scope
```

```
current_revision_id
created_at
created_by

object_revision
---------------
id                     immutable revision ID
object_id
revision_number
schema_version
epistemic_kind
payload                JSONB
created_at
created_by_actor_id
supersedes_revision_id

relationship
------------
id
subject_object_id
predicate              evolving string/type identifier
object_object_id
subject_revision_id    nullable
object_revision_id     nullable
qualifiers             JSONB
created_at
created_by_activity_id

activity
--------
id
kind                   import / human_edit / derive / model_infer / ...
actor_id
tool_name
tool_version
model_name              nullable
model_version           nullable
configuration           JSONB
configuration_digest    nullable
started_at
ended_at

activity_input
--------------
activity_id
input_revision_id
role
locator                 JSONB
```

```
activity_output
---------------
activity_id
output_revision_id
role

evidence_link
-------------
id
knowledge_revision_id
source_object_id
source_revision_id
evidence_role
locator                JSONB
created_by_activity_id

artifact_ref
------------
id
object_id              nullable
revision_id            nullable
uri
content_digest
media_type
size
external_version       JSONB
created_at
```

Primary and foreign keys give the core references database-enforced integrity. PostgreSQL's documented foreign-key mechanism directly supports self-referential and many-to-many structures, so no graph store is required to make these objects relational. [34]

The crucial design decision is that **a revision should be immutable after publication**. An edit creates a new revision and atomically changes `current_revision_id`. That is an application-level Shiroe rule, not a PostgreSQL feature. It follows the successful revision-history precedent in Invenio while making provenance edges safe against later mutation. [11]

`kind`, `predicate`, and JSONB payloads should initially be open-ended. The database envelope freezes identity/provenance semantics, not the ontology.

Search projection

Initial lexical indexing can remain inside PostgreSQL:

```
search_document
---------------
object_revision_id
search_text
search_vector          tsvector
projection_version
generated_at
```

A GIN index on `search_vector` is PostgreSQL's preferred full-text index arrangement. [73] Keeping the projection separately identifiable is preferable to treating `tsvector` as mysterious internal state because it gives Shiroe a place to record which extraction/projection version produced the searchable text.

If vectors are later justified:

```
embedding
---------
object_revision_id
chunk_or_field
model
model_version
dimensions
vector
source_digest
created_at
```

The vector is never the record. It is a derivative of a particular revision using a particular model. pgvector can keep such embeddings transactionally associated with their source metadata and supply exact or ANN lookup. [68]

Shared writes and consistency

Every mutation should go through an API operation with an expected revision:

```
update_object(
    object_id,
    expected_revision_id,
    new_payload,
    provenance
)
```

If the current revision no longer equals the caller's expected revision, the server rejects or explicitly merges the operation rather than silently overwriting another agent's work. PostgreSQL supplies the underlying transaction/locking semantics, while this revision check gives Shiroe domain-level optimistic concurrency. [61]

Write APIs should also accept an idempotency token. Letta's current client/server protocol provides a useful precedent for sequencing and idempotency in agent-driven operations. [26] This matters disproportionately for HPC login nodes, flaky network connections, agent retries, and duplicate tool calls: retrying “record this interpretation” must not create three indistinguishable interpretations.

The server is the only owner of database credentials in normal operation. CLI, MCP clients, agents, laptops, workstations, and HPC nodes authenticate to Shiroe—not to PostgreSQL. This directly implements the project's documented-interface invariant. filecite turn0file0

Conditions that justify another service

The PostgreSQL hypothesis should be falsifiable. Phase 0 should write down gates now rather than debating them later.

Add pgvector when a representative retrieval benchmark shows dense or hybrid retrieval materially improves a Shiroe-relevant quality metric over lexical search alone, and PostgreSQL can still meet the target latency/concurrency. pgvector already supplies exact/HNSW/IVFFlat search, so “we need vectors” alone is not a reason to create a vector service. [68]

A reasonable *project decision gate*, explicitly not a documented database limit, would be something like **≥5 percentage points Recall@K or ≥10% relative nDCG improvement** on a representative benchmark. Below that, operational simplicity wins.

Add OpenSearch when at least one of these is demonstrated:

- required fuzzy, analyzer, multi-field, faceted or hybrid-ranking behavior is awkward/unavailable in the PostgreSQL implementation and improves measured retrieval quality;
- search P95 exceeds the project's target after reasonable PostgreSQL indexing/tuning;
- search/index maintenance measurably harms canonical write latency or database I/O;
- search must scale or fail independently from transactional state;
- vector scale/memory requirements exceed what the tested pgvector configuration can economically provide.

OpenSearch's documented Lucene text features, hybrid pipelines/RRF, vector search, disk-oriented vector options, and independent reindex/snapshot mechanisms make it a credible next step under those conditions. [74]

For the Phase 0 prototype, the performance gate could be set before testing—for example **P95 retrieval under 200 ms at the intended concurrency**—so the architecture does not move goalposts after seeing results. That 200 ms figure is a proposed Shiroe service objective, not a PostgreSQL or OpenSearch limit.

Add a graph engine only when an actual Shiroe workload requires frequent, variable-depth graph traversal and PostgreSQL recursive/relational approaches fail the agreed service objective. Relationship count alone is not sufficient evidence.

Add a dedicated object/artifact service when Shiroe itself becomes responsible for uploading, access-controlling, retaining and serving artifacts rather than merely referencing user-owned stores. MLflow's explicit artifact-server/backend-store separation is the right precedent at that point. [75]

Add a durable job queue when indexing, extraction, embedding or other derivations become asynchronous jobs that must survive server restarts or coordinate across multiple Shiroe replicas. Until then, a new queue/Redis service creates more recovery and consistency state than Phase 0 needs.

Unresolved questions and discriminating prototype tests

Phase 0 does not need to resolve later questions about scientific document extraction, experiment execution, project-management workflow, or the final ontology. It does need to falsify the architectural assumptions above with small tests.

Unresolved question	Small test	Pass/fail signal
Can PostgreSQL serve canonical state and initial lexical retrieval without operational interference?	Generate a modest but deliberately oversized Phase 0 corpus—for example 100k–1m revision records plus relationships and textual projections. Run mixed read/write/search traffic using realistic API queries.	Record P50/P95/P99 lookup, write and FTS latency plus database CPU/I/O. Add OpenSearch only if the agreed target is missed after ordinary indexing/tuning or FTS materially harms transactional work.
Is PostgreSQL FTS good enough as the first retrieval layer?	Build perhaps 50–100 representative research-state queries over synthetic or existing project text, with human relevance judgments. Compare PostgreSQL FTS against OpenSearch lexical search; optionally add pgvector as a third arm. This is an architecture benchmark, *not* the comprehensive Phase 1 scientific-retrieval review. PostgreSQL and OpenSearch expose materially different text-search machinery, so testing rather than assuming parity is warranted. [76]	Keep PostgreSQL if quality is adequate. Introduce another search layer only for a predefined meaningful relevance gain or missing feature.
Can every derived index be destroyed safely?	Create canonical state, build FTS and optional vectors, record expected query/result fixtures, then deliberately delete every derived index/projection and regenerate them solely from canonical revisions plus versioned source material.	Stable IDs/evidence links must remain intact; reconstructed projections must reference exactly the same source revisions. Failure means Shiroe has accidentally made an index canonical. OpenSearch's explicit reindex model is the precedent if an external projection is later used. [77]

Unresolved question	Small test	Pass/fail signal
Do revision semantics survive simultaneous agents?	Run 10–20 concurrent clients against one object: stale edits, valid edits, duplicate retries and relation creation. Repeat through REST and MCP.	No silent lost update; duplicate idempotency keys create one mutation; every accepted state transition has an attributable revision/activity. PostgreSQL's concurrency mechanisms are the substrate, but this test validates Shiroe's API semantics. [61]
Is the provenance envelope expressive without becoming an ontology?	Encode five deliberately different cases: human note, direct observation, deterministic derived metric, LLM interpretation, speculative hypothesis. Give each evidence from a precise prior revision/artifact. Ask the API to explain lineage in both directions.	A client must be able to distinguish all five without inspecting prose, and trace each derived/inferred result to exact inputs and generation activity. W3C PROV provides the underlying entity/activity/agent precedent. [29]
Can schema evolution preserve old projects?	Start with schema version A; ingest objects; add a new payload field and a new relationship type in version B; migrate; keep old API fixtures. Then export and restore the mixed-version database.	Stable IDs and old revisions remain valid; old clients either continue to work or receive a deliberately versioned API response. No migration rewrites historical evidence merely to match the current ontology. Invenio's explicit UUID/revision/JSON-schema pattern makes this a particularly relevant precedent. [11]
Are external artifacts reproducibly addressable from different machines?	Register examples from Git, ordinary filesystem/shared HPC storage, and one content-addressed workflow such as DataLad or DVC. Resolve references from laptop and HPC login node; intentionally change a file at the same path.	Shiroe detects changed content/version rather than silently treating the path as the same artifact. DataLad's separation of dataset, version and content IDs provides the model to emulate. [44]
Is Shiroe recoverable without its retrieval services?	Take a PostgreSQL logical backup and a portable Shiroe export. Destroy the deployment. Restore PostgreSQL, reconnect artifact references, rebuild FTS and any optional vectors from scratch.	Stable object/revision IDs, relationships, provenance and artifact references match pre-failure fixtures; no search snapshot is necessary. PostgreSQL explicitly supports logical dump/restore for this recovery path. [62]

Unresolved question	Small test	Pass/fail signal
Does MCP remain a thin adapter rather than a second architecture?	Implement the same `get`, `search`, `create-revision`, `relate`, and `explain-provenance` operations over REST and MCP and run common contract tests.	Both transports produce equivalent application results and share authorization/concurrency semantics. MCP's explicit resource/tool and identifier-oriented model makes this feasible without moving state into the protocol layer. [60]
Can a real distributed Phase 0 deployment meet the project's completion gate?	Run one Shiroe server and access the same records from a workstation, laptop and HPC login node using at least CLI, one generic MCP client and one agent harness. Disconnect/ reconnect clients and repeat writes.	All clients observe the same stable IDs/revisions through documented interfaces, with no machine-local state required for correctness—the completion condition already specified by the development plan. filecite turn0file0

The tests deliberately avoid answering Phase 1's questions about paper ingestion, sophisticated scientific retrieval, claim extraction or literature APIs, and Phase 3's questions about experiment execution. The objective is only to determine whether the **persistence substrate and service boundary are trustworthy enough for those later layers to build upon**.

The resulting Phase 0 architecture is therefore deliberately boring in the best possible way: **PostgreSQL owns canonical Shiroe research state; external systems own heavyweight artifact bytes; Git identities are preserved rather than replicated; provenance and revisions are explicit; FTS and vectors are disposable projections; REST is the ordinary machine interface; MCP is an agent adapter; and every additional infrastructure service has to earn its existence through a failed benchmark or an unmet capability.** That architecture incorporates the strongest recurring lessons from scientific repositories, experiment trackers, agent-memory systems, provenance standards, search engines and Git-backed artifact systems while preserving Shiroe's more unusual—and scientifically important—requirement that *what happened, what a human said, what was computed, what a model inferred, and what is merely hypothesized never collapse into the same kind of "memory."* filecite turn0file1

---

[1] [8] [53] Preparation — Dataverse.org
https://guides.dataverse.org/en/6.7.1/installation/prep.html?utm_source=chatgpt.com

[2] [38] PostgreSQL: Documentation
https://www.postgresql.org/docs/?utm_source=chatgpt.com

[3] PostgreSQL: PostgreSQL 18 Press Kit
https://www.postgresql.org/about/press/presskit18/en/?utm_source=chatgpt.com

[4] [35] PostgreSQL: Documentation: 18: 8.14. JSON Types
https://www.postgresql.org/docs/18/datatype-json.html?utm_source=chatgpt.com

[5] [29] [55] PROV-DM: The PROV Data Model
https://www.w3.org/TR/prov-dm/?utm_source=chatgpt.com

6 23 Letta App Server | Letta Docs
https://docs.letta.com/self-hosting/app-server?utm_source=chatgpt.com

7 27 GitHub - mem0ai/mem0: The Memory Layer for AI Agents - Drop-in memory infrastructure for AI agents and apps. Context that persists. Built for production. · GitHub
https://github.com/mem0ai/mem0?utm_source=chatgpt.com

9 Backups — Dataverse.org
https://guides.dataverse.org/en/latest/admin/backups.html?utm_source=chatgpt.com

10 GitHub · Where software is built
https://github.com/IQSS/dataverse/milestone/124?utm_source=chatgpt.com

11 57 69 GitHub - inveniosoftware/invenio-records: Invenio-Records is a metadata storage module. · GitHub
https://github.com/inveniosoftware/invenio-records?utm_source=chatgpt.com

12 GitHub - inveniosoftware/invenio-rdm-records: DataCite-based data model for InvenioRDM flavour. · GitHub
https://github.com/inveniosoftware/invenio-rdm-records?utm_source=chatgpt.com

13 Invenio digital repository framework · GitHub
https://github.com/inveniosoftware?utm_source=chatgpt.com

14 MLflow Tracking Server | MLflow AI Platform
https://www.mlflow.org/docs/latest/self-hosting/architecture/tracking-server/?utm_source=chatgpt.com

15 Artifact Stores | MLflow AI Platform
https://www.mlflow.org/docs/latest/self-hosting/architecture/artifact-store/?utm_source=chatgpt.com

16 54 75 Backend Stores | MLflow AI Platform
https://mlflow.org/docs/latest/tracking/backend-stores/?utm_source=chatgpt.com

17 MLflow · GitHub
https://github.com/mlflow?utm_source=chatgpt.com

18 Overview — Aim 3.29.1 documentation
https://aimstack.readthedocs.io/en/latest/understanding/overview.html?utm_source=chatgpt.com

19 Aim CLI — Aim 3.29.1 documentation
https://aimstack.readthedocs.io/en/latest/refs/cli.html?utm_source=chatgpt.com

20 GitHub - aimhubio/aim: Aim 💫 — An easy-to-use & supercharged open-source experiment tracker. · GitHub
https://github.com/aimhubio/aim?utm_source=chatgpt.com

21 Unauthenticated tracking RPC can invoke Repo.rm to delete arbitrary Aim repositories · Issue #3422 · aimhubio/aim · GitHub
https://github.com/aimhubio/aim/issues/3422?utm_source=chatgpt.com

22 Stateful agents | Letta Docs
https://docs.letta.com/concepts/stateful-agents?utm_source=chatgpt.com

24 Attaching and detaching memory blocks | Letta Docs
https://docs.letta.com/tutorials/attaching-detaching-blocks/?utm_source=chatgpt.com

25 Letta · GitHub
https://github.com/letta-ai?utm_source=chatgpt.com

26 Protocol lifecycle | Letta Docs
https://docs.letta.com/self-hosting/app-server/protocol-lifecycle?utm_source=chatgpt.com

28 GitHub - mem0ai/mem0-mcp · GitHub
https://github.com/mem0ai/mem0-mcp?utm_source=chatgpt.com

30 Wikibase: JSON
https://doc.wikimedia.org/Wikibase/master/php/docs_topics_json.html?utm_source=chatgpt.com

31 Wikibase: Constraints
https://doc.wikimedia.org/Wikibase/master/php/docs_topics_constraints.html?utm_source=chatgpt.com

32 GitHub - wikimedia/mediawiki-extensions-Wikibase: 🌐 The knowledge base software that drives Wikidata.org. Mirror from https://gerrit.wikimedia.org/g/mediawiki/extensions/Wikibase. See https://www.mediawiki.org/wiki/Developer_access for contributing. · GitHub
https://github.com/wikimedia/mediawiki-extensions-Wikibase?utm_source=chatgpt.com

33 mediawiki-extensions-WikibaseQualityConstraints/extension.json at master · wikimedia/mediawiki-extensions-WikibaseQualityConstraints · GitHub
https://github.com/wikimedia/mediawiki-extensions-WikibaseQualityConstraints/blob/master/extension.json?utm_source=chatgpt.com

34 64 PostgreSQL: Documentation: 18: 5.5. Constraints
https://www.postgresql.org/docs/current/ddl-constraints.html?utm_source=chatgpt.com

36 PostgreSQL: Documentation: 18: 9.13. Text Search Functions and Operators
https://www.postgresql.org/docs/18/functions-textsearch.html?utm_source=chatgpt.com

37 61 PostgreSQL: Documentation: 16: 13.1. Introduction
https://www.postgresql.org/docs/16/mvcc-intro.html?utm_source=chatgpt.com

39 68 GitHub - pgvector/pgvector: Open-source vector similarity search for Postgres · GitHub
https://github.com/pgvector/pgvector?utm_source=chatgpt.com

40 pgvector/LICENSE at master · pgvector/pgvector · GitHub
https://github.com/pgvector/pgvector/blob/master/LICENSE?utm_source=chatgpt.com

41 70 74 Full-text queries | OpenSearch Documentation
https://docs.opensearch.org/latest/query-dsl/full-text/index/?utm_source=chatgpt.com

42 Take and restore snapshots | OpenSearch Documentation
https://docs.opensearch.org/latest/tuning-your-cluster/availability-and-recovery/snapshots/snapshot-restore/?utm_source=chatgpt.com

43 GitHub - opensearch-project/OpenSearch: 🔎 Open source distributed and RESTful search engine. · GitHub
https://github.com/opensearch-project/opensearch?utm_source=chatgpt.com

44 67 Basic principles — DataLad 1.6.3+38.g65cd1aa documentation
https://docs.datalad.org/en/latest/basics.html?utm_source=chatgpt.com

45 File URL handling — DataLad 1.6.3+38.g65cd1aa documentation
https://docs.datalad.org/en/latest/design/file_url_handling.html?utm_source=chatgpt.com

46 GitHub - datalad/datalad: Keep code, data, containers under control with git and git-annex · GitHub
https://github.com/datalad/datalad?utm_source=chatgpt.com

47 58 Remote Storage | Data Version Control · DVC
https://doc.dvc.org/user-guide/data-management/remote-storage?utm_source=chatgpt.com

48 Discovering and Accessing Data | Data Version Control · DVC
https://doc.dvc.org/user-guide/data-management/discovering-and-accessing-data?utm_source=chatgpt.com

49 pull | Data Version Control · DVC
https://doc.dvc.org/command-reference/pull?utm_source=chatgpt.com

50 modelcontextprotocol/docs/specification/2026-07-28/index.mdx at main · modelcontextprotocol/modelcontextprotocol · GitHub
https://github.com/modelcontextprotocol/modelcontextprotocol/blob/main/docs/specification/2026-07-28/index.mdx?utm_source=chatgpt.com

51 Releases · modelcontextprotocol/modelcontextprotocol · GitHub
https://github.com/modelcontextprotocol/modelcontextprotocol/releases?utm_source=chatgpt.com

52 As part of MCP joining the Linux Foundation under the Agentic AI Foundation, we're updating our licensing to better align with LF project standards. This post explains what's changing and why. · Issue #3197 · modelcontextprotocol/modelcontextprotocol · GitHub
https://github.com/modelcontextprotocol/modelcontextprotocol/issues/3197?utm_source=chatgpt.com

56 73 PostgreSQL: Documentation: 18: 12.9. Preferred Index Types for Text Search
https://www.postgresql.org/docs/18/textsearch-indexes.html?utm_source=chatgpt.com

59 ML Experiment Tracking | MLflow AI Platform
https://www.mlflow.org/docs/latest/tracking/?utm_source=chatgpt.com

60 modelcontextprotocol/docs/specification/2026-07-28/basic/index.mdx at main · modelcontextprotocol/modelcontextprotocol · GitHub
https://github.com/modelcontextprotocol/modelcontextprotocol/blob/main/docs/specification/2026-07-28/basic/index.mdx?utm_source=chatgpt.com

62 PostgreSQL: Documentation: 18: pg_dump
https://www.postgresql.org/docs/current/app-pgdump.html?utm_source=chatgpt.com

63 Snapshots | OpenSearch Documentation
https://docs.opensearch.org/latest/tuning-your-cluster/availability-and-recovery/snapshots/index/?utm_source=chatgpt.com

65 PostgreSQL: Documentation: 18: 65.4. GIN Indexes
https://www.postgresql.org/docs/current/gin.html?utm_source=chatgpt.com

66 76 PostgreSQL: Documentation: 18: Chapter 12. Full Text Search
https://www.postgresql.org/docs/current/textsearch.html?utm_source=chatgpt.com

71 User Guide | Data Version Control · DVC
https://doc.dvc.org/user-guide?utm_source=chatgpt.com

72 MLflow Tracking Server | MLflow AI Platform
https://mlflow.org/docs/latest/self-hosting/architecture/tracking-server/?utm_source=chatgpt.com

77 Reindex Documents API | OpenSearch Documentation
https://docs.opensearch.org/latest/api-reference/document-apis/reindex/?utm_source=chatgpt.com